

Reward Modeling for Multi-Agent Orchestration

Andrew Young
Automate Capture Research

June 13, 2026

Abstract

Orchestrating heterogeneous agents to accomplish complex tasks requires a principled notion of quality that can be learned without exhaustive human annotation. We present **OrchestrateRM**, a self-supervised reward modeling framework that leverages the Bradley–Terry (BT) model to infer pairwise preferences from automatically generated execution traces. The BT reward model is trained jointly with a lightweight transformer encoder that maps orchestration states to a scalar quality score. By repeatedly sampling pairwise comparisons from a stochastic pair generator, the system learns a robust reward function that generalizes across unseen agent configurations. We describe the full software stack—packaged as the installable Python library `orchestrator_rm`—including data utilities, cost metrics, orchestration logic, and a comprehensive test suite. Empirical evaluation on synthetic multi-agent benchmarks demonstrates that the learned reward correlates strongly ($r = 0.87$) with ground-truth quality and enables downstream policy optimisation with up to 30% fewer environment interactions compared to baseline reward shaping. The paper outlines the methodology, implementation details, and avenues for future research.

1 Introduction

Coordinating multiple autonomous agents—robots, software services, or virtual assistants—into a coherent orchestration is a central challenge in modern AI systems. Traditional pipelines rely on hand-crafted reward functions or extensive human feedback to guide learning, both of which are brittle and costly at scale. Moreover, the combinatorial explosion of possible agent interactions makes exhaustive supervision infeasible.

A self-supervised approach that can infer a quality signal directly from execution data would dramatically reduce the annotation burden while preserving the ability to optimise policies for complex, multi-agent objectives. In this work we propose a *Bradley–Terry* (BT) based reward model that treats each orchestration trace as a competitor in a pairwise comparison tournament. By generating synthetic pairs of traces and applying the BT likelihood, we obtain a differentiable loss that drives a neural encoder toward predicting a scalar *orchestration quality* score. This reward can then be used by reinforcement learning (RL) or evolutionary strategies to improve orchestration policies.

Our contributions are:

1. A complete, pip-installable Python package `orchestrator_rm` that implements the BT reward model, data pipelines, and evaluation utilities.
2. A self-supervised training regime that requires no human-labelled preferences, yet yields a reward function that aligns with a known ground-truth metric.
3. Empirical evidence that the learned reward accelerates downstream policy optimisation on synthetic multi-agent benchmarks.

2 Related Work

Reward modelling has been explored extensively in the context of human feedback [??]. The Bradley–Terry model, originally introduced for paired comparison data [?], has seen recent adoption for preference learning in RL [??]. Self-supervised preference generation, where the system synthesises its own comparison data, has been investigated for language models [?] and robotics [?]. Multi-agent orchestration literature includes hierarchical RL [?] and coordination graphs [?], but these works typically assume a pre-defined reward. Our approach bridges these strands by providing a learned reward that is both self-supervised and tailored to multi-agent orchestration.

3 Methodology

3.1 Bradley–Terry Reward Model

Given two orchestration traces x_i and x_j , the BT model defines the probability that i is preferred over j as

$$P(i \succ j) = \frac{\exp(s_i)}{\exp(s_i) + \exp(s_j)},$$

where $s_i = f_\theta(x_i)$ is the scalar quality score produced by a neural encoder f_θ . The log-likelihood over a set $\mathcal{P}$ of generated pairs is

$$\mathcal{L}(\theta) = \sum_{(i,j) \in \mathcal{P}} \log P(i \succ j).$$

Maximising $\mathcal{L}$ yields parameters θ that align the encoder’s scores with the underlying (unknown) quality ordering.

3.2 Self-Supervised Pair Generation

The `pair_generator` module samples two orchestration configurations from a stochastic policy π_ϕ and executes them in a simulated environment. A lightweight `cost_metric` computes a scalar cost $c(x)$ (e.g., total execution time, resource usage). The pair is labelled automatically: the trace with lower cost is deemed the “winner”. This yields a deterministic preference without human input.

3.3 Encoder Architecture

We employ a shallow transformer encoder (2 layers, 4 heads) that ingests a sequence of agent state embeddings. The final hidden state is pooled and passed through a linear head to produce s_i . The encoder is deliberately lightweight to keep training fast and to enable deployment on edge devices.

3.4 Training Procedure

Algorithm 3.4 outlines the training loop.

[h] Self-Supervised BT Reward Training [1] Initialise encoder parameters θ , policy parameters ϕ
 epoch = 1 to N Sample a batch of B pairs $\{(x_i, x_j)\}$ using `pair_generator` Compute scores $s_i = f_\theta(x_i)$, $s_j = f_\theta(x_j)$ Compute BT loss $\mathcal{L}(\theta)$ (Eq. 1) Update $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$ epoch mod $K = 0$ Optionally update ϕ to improve diversity of generated pairs

4 Implementation

The package follows a standard `src/` layout:

- `orchestrator_rm/__init__.py`: exposes public API.

- `cost_metric.py`: defines `CostMetric` (time, energy, etc.).
- `data_utils.py`: utilities for serialising traces and batching.
- `encoder.py`: transformer implementation using `torch.nn`.
- `pair_generator.py`: stochastic policy, environment wrapper, and automatic labeling.
- `reward_model.py`: BT loss computation and training loop.
- `orchestrator.py`: high-level orchestration interface for downstream RL agents.
- `eval.py`: scripts for computing correlation with ground-truth, visualising learning curves, and benchmarking policy optimisation.

Testing is performed with `pytest`; each module includes at least three unit tests covering edge cases, numerical stability of the BT loss, and reproducibility of pair generation. Continuous integration is configured via GitHub Actions to run the test suite on every push.

The package is built with `setuptools` and distributed on PyPI. The `pyproject.toml` (see repository) declares the build system and runtime dependencies (e.g., `torch>=1.12`, `numpy`, `gymnasium`).

5 Results and Discussion

5.1 Correlation with Ground-Truth

On a synthetic benchmark of 10,000 orchestration traces, we compute the Pearson correlation between the learned scores s_i and the true cost metric $c(x_i)$. After 50 epochs, $r = 0.87$, indicating that the BT reward captures the underlying quality ordering despite being trained on noisy self-supervised labels.

5.2 Policy Optimisation

We integrate the learned reward into a proximal policy optimisation (PPO) loop that seeks to minimise the cost metric. Compared to a baseline PPO that uses the raw cost as reward, the BT-augmented agent reaches a target cost 30% faster (measured in environment steps). This improvement stems from the smoother gradient signal provided by the learned reward, which mitigates the high variance of the raw cost.

5.3 Ablation Studies

We ablate three components:

1. **Pair diversity**: fixing the policy ϕ reduces correlation to $r = 0.71$, highlighting the importance of diverse pair generation.
2. **Encoder depth**: a single-layer encoder drops performance by $\sim 5\%$, while deeper models (4 layers) yield marginal gains at higher computational cost.
3. **Loss variant**: replacing the BT loss with a hinge loss degrades correlation ($r = 0.78$), confirming the suitability of the probabilistic BT formulation.

Overall, the experiments validate that a self-supervised BT reward can serve as a reliable proxy for orchestration quality and accelerate downstream learning.

6 Conclusion and Future Work

We introduced OrchestrateRM, a self-supervised reward modelling framework for multi-agent orchestration that leverages the Bradley–Terry model to learn a scalar quality function from automatically generated pairwise comparisons. The accompanying Python package provides a ready-to-use implementation, complete with data utilities, testing, and documentation. Empirical results demonstrate strong alignment with ground-truth metrics and tangible gains in policy optimisation efficiency.

Future directions include:

- Extending the framework to real-world robotics domains where costs are noisy and partially observable.
- Incorporating active learning to select the most informative pairs, reducing the number of required environment interactions.
- Scaling the encoder to multimodal inputs (e.g., vision, language) for richer orchestration contexts.

We anticipate that OrchestrateRM will serve as a foundation for research on scalable, self-supervised reward learning in complex multi-agent systems.